

Google Test Overview

Nicholas Burgess



MHI

Nicholas.Burgess@uk.mizuho-sc.com

24th September 2015

Problem using Google Test Framework with Large Tests

- Time Consuming to Set-Up Large Tests
- Curve & Model Dependencies Make it Difficult to Price Test
- Dependencies Must be Loaded Before Price Testing

Solution: Test Recording & Playback

- A Curve & Model Dependency Framework has been Implemented
- Test Recording Features have been Added

1 How do Google Unit Tests Work?

2 Assertions

- Fatal & Non-Fatal
- Example: Fatal Test
- Assertion Types

3 Test Fixtures

- What are they?
- Syntax
- Example

4 Running Tests

- On the Console
- Using the Guitar

Google Unit Tests

Unit tests should be kept simple and typically just compare two values. Simplicity is key to keeping the implementation, maintenance & running of tests light-work. Tests should be written at the same time as source-code. Large tests should be decomposed into smaller tests using Google Mock.

```
1 #include <gTest/gtest.h> // Required for Google Test
2 #include <MyFunction.h> // Required for your function
3
4 // Use TEST Macro for Tests
5 // Argument 1 = Test Name
6 // Argument 2 = Test Description
7 TEST( AddTest, CheckIfOnePlusOneEqualsTwo )
8 {
9     EXPECT_EQ( 2, addFunction( 1, 1 ) );
10 }
```

Assertions

Assertion types can be fatal and non-fatal. Non-fatal tests do not prevent other tests from being executed. However Fatal tests abort the execution of further tests in a code block, indicating broad failure.

Example (Fatal Tests)

- To check the dimensions of a vector before performing calculations
- To ensure a Model is loaded before testing pricing functions
- To prevent price testing before a yield curve has been loaded
- To check market data correct before testing model functions

```
1 #include <gTest/gTest.h> // Required for Google Test
2 #include <MyVector.h>    // Required for your function
3
4 // Use TEST Macro for Tests
5 TEST( VectorTest, CheckVectorElements )
6 {
7     // Call your function
8     std::vector v = getVectorResults();
9
10    // Fatal Test: Do not continue if vector out of bounds
11    ASSERT_EQ( 5, v.size() );
12
13    EXPECT_EQ( 1, v[0] ) << " Value of v[0] should be 1" ;
14    EXPECT_EQ( 2, v[1] ) << " Value of v[1] should be 2" ;
15    EXPECT_EQ( 3, v[2] ) << " Value of v[2] should be 3" ;
16    EXPECT_EQ( 4, v[3] ) << " Value of v[3] should be 4" ;
17    EXPECT_EQ( 5, v[4] ) << " Value of v[4] should be 5" ;
18 }
```

Basic Assertions

Fatal assertion	Nonfatal assertion	Verifies
<code>ASSERT_TRUE(condition);</code>	<code>EXPECT_TRUE(condition);</code>	<i>condition is true</i>
<code>ASSERT_FALSE(condition);</code>	<code>EXPECT_FALSE(condition);</code>	<i>condition is false</i>

Binary Comparison

Fatal assertion	Nonfatal assertion	Verifies
<code>ASSERT_EQ(expected, actual);</code>	<code>EXPECT_EQ(expected, actual);</code>	<i>expected == actual</i>
<code>ASSERT_NE(val1, val2);</code>	<code>EXPECT_NE(val1, val2);</code>	<i>val1 != val2</i>
<code>ASSERT_LT(val1, val2);</code>	<code>EXPECT_LT(val1, val2);</code>	<i>val1 < val2</i>
<code>ASSERT_LE(val1, val2);</code>	<code>EXPECT_LE(val1, val2);</code>	<i>val1 <= val2</i>
<code>ASSERT_GT(val1, val2);</code>	<code>EXPECT_GT(val1, val2);</code>	<i>val1 > val2</i>
<code>ASSERT_GE(val1, val2);</code>	<code>EXPECT_GE(val1, val2);</code>	<i>val1 >= val2</i>

String Comparison

Fatal assertion	Nonfatal assertion	Verifies
<code>ASSERT_STREQ(expected_str, actual_str);</code>	<code>EXPECT_STREQ(expected_str, actual_str);</code>	the two C strings have the same content
<code>ASSERT_STRNE(str1, str2);</code>	<code>EXPECT_STRNE(str1, str2);</code>	the two C strings have different content
<code>ASSERT_STRCASEEQ(expected_str, actual_str);</code>	<code>EXPECT_STRCASEEQ(expected_str, actual_str);</code>	the two C strings have the same content, ignoring case
<code>ASSERT_STRCASENE(str1, str2);</code>	<code>EXPECT_STRCASENE(str1, str2);</code>	the two C strings have different content, ignoring case

Double Comparison

Fatal assertion	Nonfatal assertion	Verifies
<code>ASSERT_FLOAT_EQ(expected, actual);</code>	<code>EXPECT_FLOAT_EQ(expected, actual);</code>	the two float values are almost equal
<code>ASSERT_DOUBLE_EQ(expected, actual);</code>	<code>EXPECT_DOUBLE_EQ(expected, actual);</code>	the two double values are almost equal

By "almost equal", we mean the two values are within 4 ULP's from each other.

The following assertions allow you to choose the acceptable error bound:

Fatal assertion	Nonfatal assertion	Verifies
<code>ASSERT_NEAR(val1, val2, abs_error);</code>	<code>EXPECT_NEAR(val1, val2, abs_error);</code>	the difference between <i>val1</i> and <i>val2</i> doesn't exceed the given absolute error

Exception Tests

Fatal assertion	Nonfatal assertion	Verifies
<code>ASSERT_THROW(statement, exception_type);</code>	<code>EXPECT_THROW(statement, exception_type);</code>	<i>statement</i> throws an exception of the given type
<code>ASSERT_ANY_THROW(statement);</code>	<code>EXPECT_ANY_THROW(statement);</code>	<i>statement</i> throws an exception of any type
<code>ASSERT_NO_THROW(statement);</code>	<code>EXPECT_NO_THROW(statement);</code>	<i>statement</i> doesn't throw any exception

Death Tests¹

Fatal assertion	Nonfatal assertion	Verifies
<code>ASSERT_DEATH(statement, regex');</code>	<code>EXPECT_DEATH(statement, regex');</code>	<i>statement</i> crashes with the given error
<code>ASSERT_DEATH_IF_SUPPORTED(statement, regex');</code>	<code>EXPECT_DEATH_IF_SUPPORTED(statement, regex');</code>	if death tests are supported, verifies that <i>statement</i> crashes with the given error; otherwise verifies nothing
<code>ASSERT_EXIT(statement, predicate, regex');</code>	<code>EXPECT_EXIT(statement, predicate, regex');</code>	<i>statement</i> exits with the given error and its exit code matches <i>predicate</i>

¹These test for application failure

Test Fixtures

These are classes with embedded virtual methods allowing multiple tests to access and reuse the same object configuration. They are often used to reuse model, trade and other financial data. Test Fixtures gracefully manage faulty code and handle crashes and other bad behavior well.

Example (s)

- Perhaps we want to reuse the same model to test multiple products
- or price test the same trade using different models
- or reuse the same yield curve to test a portfolio of swaps

```
1 #include <gTest/gTest.h> // Required for Google Test
2
3 // Fixture Class
4 class FixtureTest : public testing::Test
5 {
6     protected:
7         // Constructor called by the TEST_F macro
8         FixtureTest() {};
9
10        // Destructor called when TEST_F goes out of scope
11        virtual ~FixtureTest() {};
12
13        // SetUp() is run immediately before every test
14        virtual void SetUp() {};
15
16        // TearDown() is run immediately after every test
17        virtual void TearDown() {};
18
19        // Our data that we wish to share amongst tests
20        double fixtureResult;
21};
```



Test Fixtures

- Test Fixtures are defined using `TEST_F`
- They can access all the member variables of the fixture class
- Fixture classes can be inherited to create larger tests

```
1 // Use TEST_F Macro for Test Fixtures
2 TEST_F{ FixtureTest , PerformFixtureTest }
3 {
4     // Compare expected result of 3.142 vs fixture result
5     EXPECT_DOUBLE_EQ( 3.142 , fixtureResult );
6 }
```

```
1 #include <gTest/gTest.h>
2
3 class QuickTest : public testing::Test
4 {
5     protected:
6         QuickTest() {};
7         virtual ~QuickTest() {};
8
9         virtual void SetUp() // Called when test starts
10        {
11            starts = time( NULL );
12        }
13
14        virtual void TearDown() // Called with test ends
15        {
16            const time_t ends = time( NULL );
17            EXPECT_TRUE( ends - starts <= 5 )<<"TIMEOUT >5secs" ;
18        }
19
20        time_t starts; // The UTC time (seconds)
21};
```



How are Tests Managed?

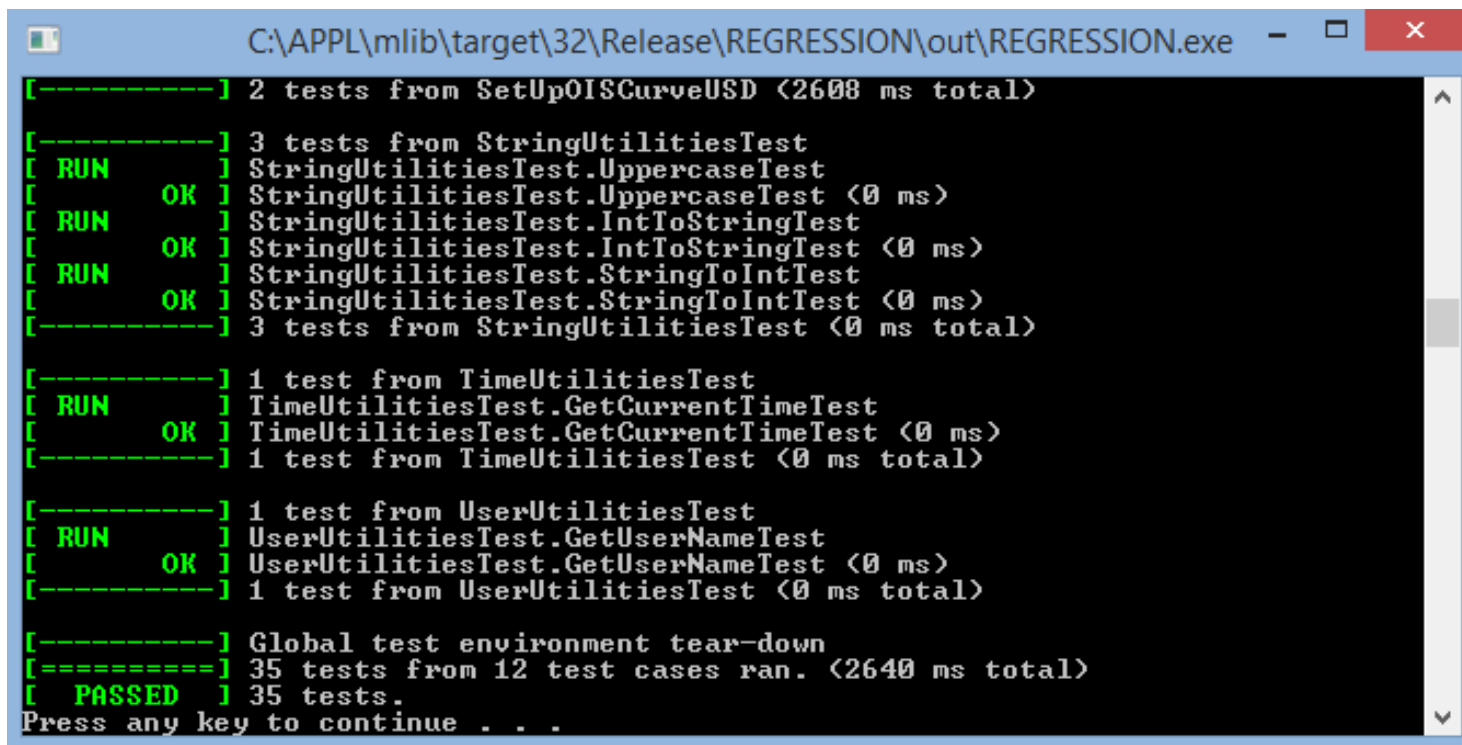
- A small Main() routine calls and runs the tests
- Google Test manages all the test declarations
- We include the Google Test Header
- Then we initialize Google Test and call RUN_ALL_TESTS

```
1 #include <gtest/gtest.h>
2 // No Need to Include Test Files
3
4 int main( int argc , char** argv )
5 {
6     // Initialize Google Test
7     testing::InitGoogleTest( &argc , argv );
8
9     // Run Tests
10    RUN_ALL_TESTS();
11 }
```

How to Launch the Tests?

- There are several ways to run the unit tests
- Tests can be run on the Console
- We can also trigger tests from a C# application called the Guitar
- Tests can be run as part of a batch build or release process
- They can also be set to run on a timer, overnight say

- The console is perfect for running and adding tests as you code.
- In MLib running tests is quick and simple
- Set the "REGRESSION" project as your start-up project
- Hit F5 to run the tests - How easy is that !!!



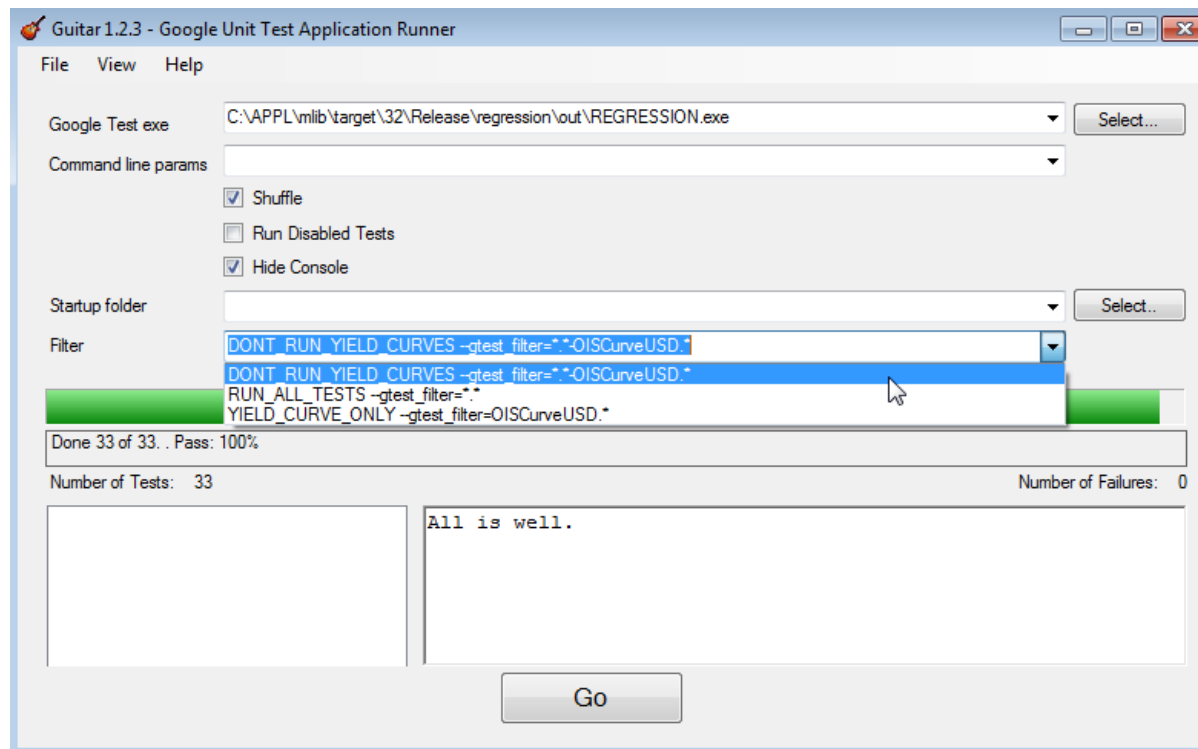
```
C:\APPL\mllib\target\32\Release\REGRESSION\out\REGRESSION.exe
[-----] 2 tests from SetUpOISCurveUSD <2608 ms total>
[-----] 3 tests from StringUtilitiesTest
[ RUN    ] StringUtilitiesTest.UppercaseTest
[ OK     ] StringUtilitiesTest.UppercaseTest <0 ms>
[ RUN    ] StringUtilitiesTest.IntToStringTest
[ OK     ] StringUtilitiesTest.IntToStringTest <0 ms>
[ RUN    ] StringUtilitiesTest.StringToIntTest
[ OK     ] StringUtilitiesTest.StringToIntTest <0 ms>
[-----] 3 tests from StringUtilitiesTest <0 ms total>
[-----] 1 test from TimeUtilitiesTest
[ RUN    ] TimeUtilitiesTest.GetCurrentTimeTest
[ OK     ] TimeUtilitiesTest.GetCurrentTimeTest <0 ms>
[-----] 1 test from TimeUtilitiesTest <0 ms total>
[-----] 1 test from UserUtilitiesTest
[ RUN    ] UserUtilitiesTest.GetUserNameTest
[ OK     ] UserUtilitiesTest.GetUserNameTest <0 ms>
[-----] 1 test from UserUtilitiesTest <0 ms total>
[-----] Global test environment tear-down
[=====] 35 tests from 12 test cases ran. (2640 ms total)
[ PASSED ] 35 tests.
Press any key to continue . . .
```

The Guitar is a C# GUI for running tests

C:\APPL\library\gtest-gbar-master\gtest-gbar-master\src bin\Release\Guitar.exe

Features Include:

- Runs as a stand-alone app or be triggered as part of a build process
- Can be configured to save test run history
- Works on both Windows and Linux (under Mono)



Google Test exe:

This is the path to the Google Test exe built by the REGRESSION project

`C:\APPL\mlib\target\32\Release\regression\out\REGRESSION.exe`

Test Filtering:

- We can select what tests to run (or not run) using filters
`LABEL -gtest_filter=TestName.TestDesc-TestName.TestDesc`
- Symbols: `*` is the wildcard, `-` indicates tests not to run
- Filters can be saved by selecting **File** then **Save Configuration**

Example (s)

```
RUN_ALL_TESTS -gtest_filter=*.*
```

```
YIELD_CURVE_ONLY -gtest_filter=OISCurveUSD.*
```

```
DONT_RUN_YIELD_CURVES -gtest_filter=*. *-OISCurveUSD.*
```

The End